NOVA CASA · TELEMETRÍA IOT EN SALESFORCE

# Arquitectura y Diseño

*Sprint 2 · Juan Diego Velásquez · John Alejandro Pastor · Octubre 2026*

## Introducción

Este documento describe la arquitectura de la solución. Empieza por el problema de Nova Casa y por qué este diseño lo resuelve; después muestra cómo se relaciona con su entorno, qué piezas la componen y cómo se organizan, cómo se comporta cuando llega una señal o cuando una persona actúa, dónde se ejecuta, cómo se protege y cómo se construye y despliega. Cierra con las decisiones de diseño, los atributos de calidad que la arquitectura asegura y los riesgos operativos con su mitigación.

Es la versión final del entregable 3 de MDSS y reemplaza, para la entrega, al diagrama de Discovery ( `entregables/03-diagrama-solucion.html` ).

> **TEXTO DE MDSS · ENTREGABLE 3**
>
> Arquitectura de una página: simulador, recepción, procesamiento, almacenamiento, intervención y consulta del operador. Marcar qué es síncrono o asíncrono y dónde se aplica seguridad.

La arquitectura se presenta en vistas complementarias, siguiendo el modelo C4 para el contexto y los componentes, y las vistas lógica, de procesos, física y de desarrollo del modelo 4+1:

| Vista | Qué responde | Sección |
|---|---|---|
| Del problema al diseño | Qué exige el problema, qué decisión responde a cada exigencia y por qué funciona | Antes de la 1 |
| Contexto | Con quién y con qué sistemas se relaciona la solución | 1 |
| Componentes | Qué piezas hay y cómo viaja una señal entre ellas | 3 |
| Lógica en capas | Cómo se organizan las piezas y qué depende de qué | 4 |
| Comportamiento (procesos) | Qué pasa en el tiempo: secuencias, decisiones y estados | 5 |

| Vista | Qué responde | Sección |
|---|---|---|
| Despliegue (física) | Dónde corre cada pieza | 6 |
| Seguridad | Con qué identidad corre cada parte y quién ve qué | 7 |
| Desarrollo | Cómo se organiza el código y cómo llega a la org | 8 |

Datos del proyecto:

- Repositorio: github.com/jpastorSFDC/nova-casa-telemetria, rama `main`.
- Org de trabajo: `nova-cdo` (Enterprise Edition).
- El detalle de clases, campos y permisos está en la **Documentación Técnica**, el **Modelo de Datos** y **Permisos de Acceso**; el comportamiento visto por cada persona, en la **Documentación Funcional**. Las decisiones están en `docs/decisiones.md`.

# Del problema al diseño

Antes de las vistas, esta sección explica por qué la solución tiene esta forma y por qué funciona como respuesta al problema de Nova Casa.

## El problema

> **TEXTO DE MDSS · BRIEF DEL CLIENTE**
> Los dispositivos pueden enviar datos repetidos, incompletos o atrasados. Una señal equivocada no debe generar trabajo duplicado ni reemplazar información más reciente. Tampoco basta con almacenar información técnica: el operador debe reconocer qué edificio necesita atención, qué tan grave es la situación y qué puede hacer después.

Hoy el operador junta la información de varios portales y correos, y a veces se entera de una falla cuando los residentes ya se quedaron sin agua. Centralizar las señales en Salesforce resuelve eso solo si lo que llega es confiable: un sensor que repite un aviso, que manda una lectura vieja después de una nueva o que manda datos inválidos no puede producir intervenciones de más ni dejar una pantalla que miente. De ahí salen cuatro exigencias que gobiernan el diseño:

1. **La señal tiene que llegar de verdad y no perderse** (BR-201, BR-202). El simulador del proveedor solo se consulta por API, no envía nada por su cuenta.
2. **Repetir, atrasar o contradecir una señal no puede cambiar el estado ni abrir trabajo de más** (BR-203, BR-205 y los acuerdos de MDSS).

3. **Lo que decide qué es grave tiene que poder cambiar sin tocar el código, y cada señal tiene que poder explicarse** (BR-204, BR-206, BR-209).
4. **Cada persona ve y hace solo lo suyo, incluso si pide los datos directamente al servidor, y la ingesta no depende de ninguna de ellas** (BR-207, BR-208).

## Cómo responde el diseño a cada exigencia

| Lo que exige el problema | Decisión de diseño | Por qué funciona | Dónde se comprueba |
|---|---|---|---|
| Recibir las señales del simulador sin captura manual, hasta 200 por tanda (BR-201). | `TelemetriaIngesta` consulta al simulador con sesión y cursor, y publica cada mensaje como Platform Event `Senial_Sensor__e`. | El simulador no empuja datos, así que alguien tiene que ir por ellos. Publicar el evento recibe Platform Events reales y separa lo que cambia con el proveedor (su contrato) de lo que cambia con el negocio (las reglas). | Corrida `QA_200` del 27 de octubre (**Documentación Funcional, sección 8**, BR-201). |
| Que publicar no se confunda con procesar (acuerdo de MDSS). | Una fila Publicada en `Log_Senial__c` se escribe antes de publicar el evento; el resultado se escribe después, al procesarlo. | Si un evento se publica y nunca se procesa, queda una fila Publicada que lo delata: el equipo distingue «llegó» de «se aplicó». | Las corridas del 27 de octubre terminaron sin ninguna fila en Publicada (**Documentación Técnica, sección 9**). |
| Un lote de hasta 200 con datos inválidos no frena a los válidos (BR-202). | Suscriptor Apex que decide el lote en memoria con un número fijo de consultas y escrituras, con aislamiento por evento y éxito parcial. | El costo no crece con el volumen. Un mensaje inválido queda Rechazado con su motivo y los demás siguen. | Atributos «Rendimiento» y «Tolerancia a fallos» (**sección 10**); BR-202 en **Documentación Funcional, sección 8**. |
| Un reenvío no abre otra intervención; la misma clave con otro contenido es un conflicto (BR-205). | Identidad `source\|messageId` única en la base de datos, más una huella SHA-256 del contenido. | La unicidad la impone la base y no el código: aun con dos lotes a la vez, un mismo mensaje nunca genera dos filas ni dos intervenciones. La huella distingue un reenvío igual de uno con otro contenido. | Atributo «Integridad» (**sección 10**); corrida `DUPLICATES` (**Documentación Funcional, sección 8**, BR-205). |

| Lo que exige el problema | Decisión de diseño | Por qué funciona | Dónde se comprueba |
|---|---|---|---|
| Una lectura de las 10:02 que llega después de la de las 10:05 no reemplaza el estado actual; una temperatura no tapa una presión (BR-203). | `Lectura_Vigente__c`, una fila por activo y tipo de medición. Gana la fecha de origen más reciente, con un desempate determinista; la severidad del activo es la mayor de sus lecturas vigentes. | El estado actual se decide por cuándo ocurrió la señal y no por cuándo llegó, y por cada medición por separado. Una lectura vieja queda como evidencia (Atrasada) sin tocar el estado. | Atributo «Consistencia temporal» (**sección 10**). |
| Una crítica nueva produce una intervención operativa, sin duplicados (BR-205, BR-206). | Solo la crítica ganadora llega a `IntervencionService`, que bloquea el activo (`FOR UPDATE`) antes de abrir el `Case` o de enlazarse al que ya está abierto. | Una crítica atrasada nunca gana, así que nunca abre una intervención. El bloqueo evita que dos lotes simultáneos abran dos para el mismo activo. | Corrida `DUPLICATES`: una intervención por activo con crítica, sin dos abiertas en el mismo (**Documentación Funcional, sección 8**, BR-205). |
| Los límites cambian sin desplegar código (BR-204). | `Umbral__c` como datos, con validación de rangos al guardar. | La persona autorizada edita el límite y la siguiente señal se clasifica con él; lo ya clasificado no se reescribe. | Atributo «Modificabilidad» (**sección 10**). |
| Cada persona ve y hace solo lo suyo, también si pide los datos al servidor (BR-208). | Compartición privada con un operador dueño por edificio y jerarquía de roles; controladores `with sharing` y `WITH USER_MODE`; permisos de acción verificados en el servidor; identidades propias para la ingesta y el procesamiento. | La restricción está en el servidor y no en la pantalla: ocultar un botón no es seguridad. La ingesta corre con su propia identidad, así que no depende de que exista un operador con permisos. | Atributos «Seguridad» e «Independencia de la ingesta» (**sección 10**). |

| Lo que exige el problema | Decisión de diseño | Por qué funciona | Dónde se comprueba |
|---|---|---|---|
| El operador reconoce qué necesita atención y el administrador entiende qué pasó con cada señal (BR-207, BR-209). | La pantalla lee lo que el procesamiento ya decidió (lectura vigente, severidad del activo, intervención); `Log_Senial__c` guarda resultado, motivo y fechas por señal. | La interfaz no recalcula nada y por eso muestra exactamente lo que el sistema decidió; investigar una señal no obliga a reconstruirla. | Atributos «Usabilidad» y «Trazabilidad» (sección 10). |

Las alternativas que se descartaron en cada caso y lo que cuestan están en la sección 9.

## La historia de Laura, a través del diseño

MDSS plantea una historia de referencia: Laura es operadora de dos edificios; una bomba reporta presión baja y ella debe identificar el edificio, entender la severidad y abrir la intervención. Si el proveedor reenvía el mismo aviso no debe haber una segunda intervención, y si después llega una lectura antigua la pantalla no debe mostrarla como la condición actual. Así la resuelve el diseño:

1. Un job programado, cada 5 minutos, o el botón «Traer señales» hace que `TelemetriaIngesta` pida al simulador el mensaje de la bomba.
2. Escribe su fila Publicada, con edificio, activo, medición, valor, unidad, fecha de origen y clave, y publica el evento `Senial_Sensor__e`.
3. El bus de eventos entrega el evento al suscriptor, junto con otros hasta completar un lote de 200.
4. `SenialSensorHandler` comprueba la identidad: si el mensaje ya se procesó con el mismo contenido, solo cuenta un reenvío, y si ya se procesó con otro contenido, lo cuenta como conflicto. En ninguno de los dos casos cambia el estado ni abre una intervención.
5. Si es un mensaje nuevo, lo clasifica con el umbral del tipo de activo y de medición. La presión baja de la bomba resulta Crítica.
6. Lo compara con la lectura vigente de esa bomba y esa medición. Si es más reciente, gana: se actualiza la lectura vigente y la severidad del activo. Si es más vieja, queda Atrasada, como evidencia, sin cambiar el estado actual.
7. La crítica ganadora llega a `IntervencionService`, que bloquea el activo y abre el `Case` En curso, o enlaza la señal al que ya estaba abierto.
8. La fila del registro pasa a su resultado final, con su hora de procesamiento y, si abrió una intervención o se enlazó a una, su vínculo a ella.
9. Laura abre la pantalla y ve solo sus dos edificios: el activo en crítico, qué tan antigua es la información, y desde ahí abre el activo o la intervención.

El reenvío del mismo aviso se detiene en el paso 4; la lectura antigua queda Atrasada en el paso 6 y la pantalla no la presenta como actual. Esas dos condiciones de la historia de Laura no dependen de la buena voluntad del código sino de la identidad única, la comparación por fecha de origen y el bloqueo del activo.

### Por qué funciona, y hasta dónde

- **Las garantías están en la estructura, no en que el código se porte bien.** La identidad única la impone la base de datos, el bloqueo del activo lo impone `FOR UPDATE`, el estado actual se decide por la fecha de origen y no por el orden de llegada, y los permisos se verifican en el servidor. Por eso siguen valiendo con 200 señales, con dos lotes a la vez o con señales que llegan fuera de orden.
- **Cada pieza responde a una exigencia y se puede cambiar sin tocar las demás.** El Platform Event separa lo que cambia con el proveedor (la recepción) de lo que cambia con el negocio (las reglas); los límites son datos; la pantalla solo lee lo ya decidido.
- **Se comprobó, no solo se afirma.** Cada exigencia tiene una prueba automatizada que falla si el comportamiento se rompe (**sección 10**) y se vio contra el simulador real el 27 de octubre de 2026 (**Documentación Técnica, sección 9**).
- **Lo que el diseño no hace.** La recepción es por sondeo y no en tiempo real: con la ingesta programada una señal puede tardar unos 5 minutos en llegar a la pantalla, más el retraso del programador en una org de prueba, y «Traer señales» la adelanta a pedido. Una intervención de telemetría abierta por activo muestra la crítica más reciente y no cada mensaje (D039). Y depende de que el simulador responda como se supuso ante errores y vencimiento de sesión (**sección 11**).

## 1. Contexto del sistema

Nova Casa Telemetría vive dentro de una org de Salesforce. Las cuatro personas de MDSS la usan desde Lightning Experience; la solución consulta al simulador del proveedor, que expone una API REST con sesión, telemetría paginada por cursor y el catálogo de edificios y activos. El simulador no empuja datos: la solución los pide.

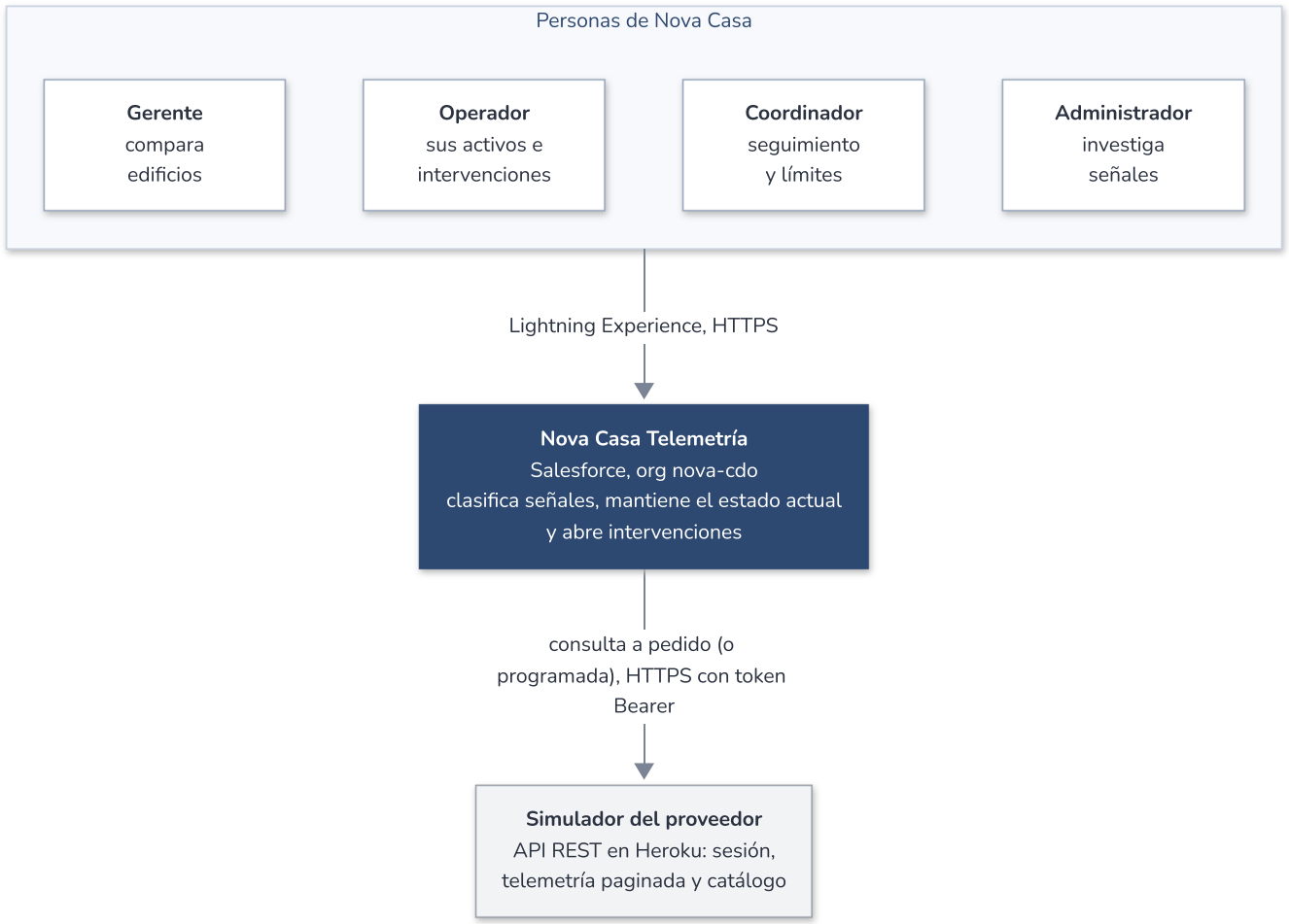



*Diagrama de contexto (C4, nivel 1).*

## 2. Estilos y patrones arquitectónicos

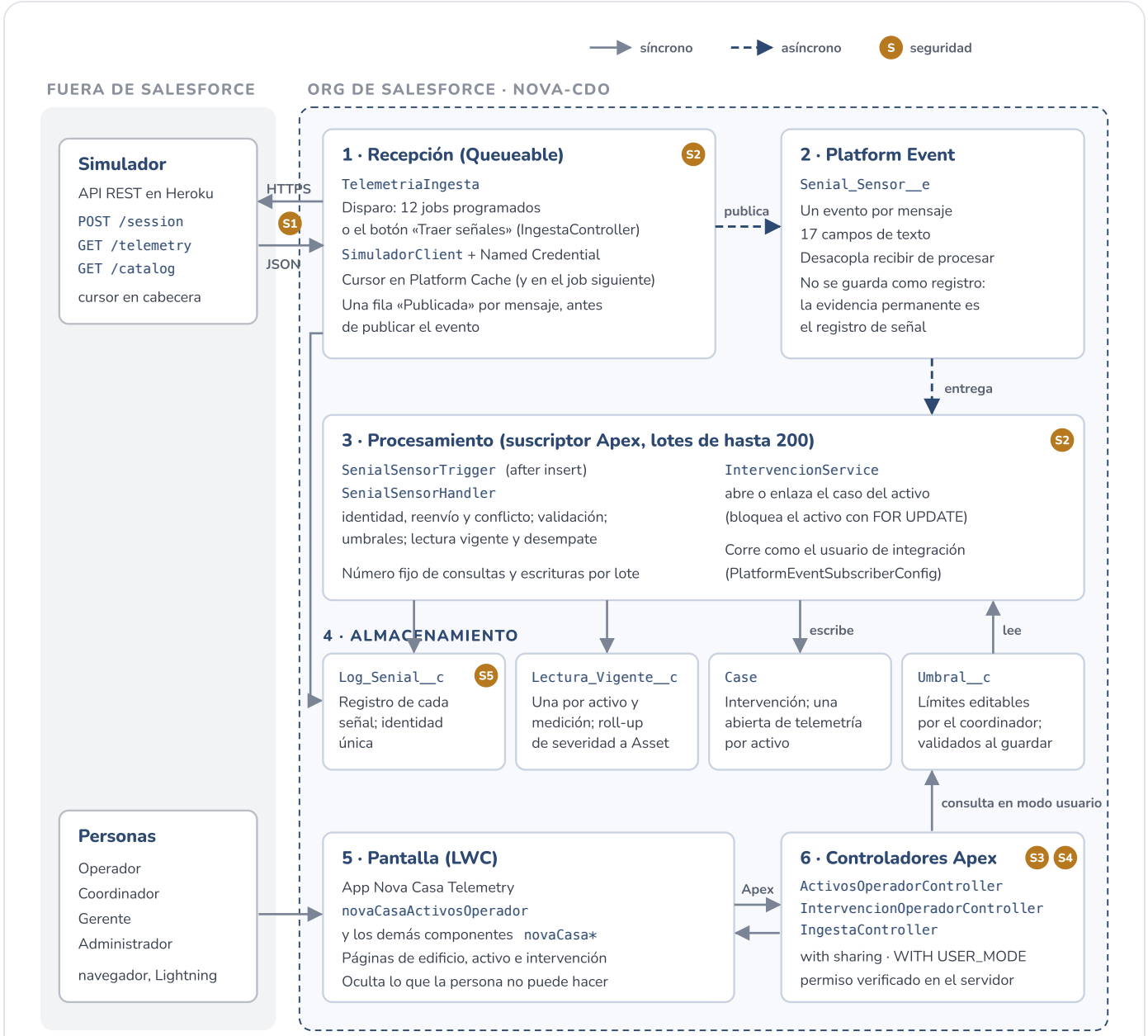
| Estilo o patrón | Dónde se aplica | Por qué |
|---|---|---|
| **Arquitectura orientada a eventos** (publicar y suscribir) | El Platform Event `Senial_Sensor__e` separa la recepción del procesamiento. | La pieza que conoce el contrato del proveedor y la que conoce las reglas cambian por separado, y publicar y procesar quedan como dos momentos con evidencia propia. |
| **Consumidor por sondeo** (polling consumer) | `TelemetriaIngesta` consulta al simulador a pedido o de forma programada cada 5 minutos, continuando la sesión con un cursor. | El simulador solo se consulta por API; no ofrece envíos hacia la org. |

| Estilo o patrón | Dónde se aplica | Por qué |
|---|---|---|
| **Receptor idempotente** | Identidad `source\|messageId` única en el registro de señal y en la intervención, más una huella del contenido. | Un reenvío no repite efectos y una misma clave con otro contenido se reconoce como conflicto. |
| **Arquitectura en capas** | Presentación, controladores, dominio, integración y datos (sección 4). | Cada capa depende de las inferiores y de las de su misma capa, nunca de las superiores; la interfaz nunca toca el procesamiento. |
| **Reglas como datos** | Los límites son registros de `Umbral__c`, con validación al guardar. | Una persona autorizada cambia los límites sin desplegar código (BR-204). |
| **Procesamiento por lotes** | El suscriptor decide un lote de hasta 200 eventos en memoria, con un número fijo de consultas y escrituras. | Respeta los límites de la plataforma sin importar el volumen. |
| **Aislamiento de fallos** | Cada evento se protege por separado y las escrituras admiten éxito parcial. | Una señal inválida o un fallo puntual no detienen al resto del lote (BR-202). |
| **Reintento con espera** | Ante 429 o 5xx la ingesta reintenta la página con 1 y 2 minutos de espera. | Tolera fallas transitorias del proveedor sin perder lo ya registrado. |
| **Bloqueo pesimista** | El activo se bloquea con `FOR UPDATE` antes de abrir o enlazar su intervención. | Dos lotes simultáneos no abren dos intervenciones para el mismo activo. |
| **Fachada para la interfaz** | Los controladores `@AuraEnabled` devuelven vistas propias, no registros completos. | La pantalla recibe solo lo que necesita y nunca campos técnicos como la identidad o la huella. |
| **Bitácora de mensajes** (message store) | `Log_Senial__c` guarda una fila por señal con su resultado, motivo y fechas. | Trazabilidad persistente para investigar y decidir si reintentar (BR-209). |

| Estilo o patrón | Dónde se aplica | Por qué |
| --- | --- | --- |
| **Mínimo privilegio** | Permission sets de negocio por persona (más `Nova_Casa_Lightning` para quienes usan la interfaz), identidades separadas para la ingesta y el procesamiento, y un administrador de la telemetría que no administra la org. | Cada parte tiene solo los permisos que su función necesita (BR-208). |

## 3. Vista de componentes

El diagrama muestra el recorrido pedido por MDSS (simulador, recepción, procesamiento, almacenamiento, intervención y consulta del operador), con el Platform Event entre la recepción y el procesamiento; la intervención es el `Case` del almacenamiento, que abre `IntervencionService`. También muestra qué pasos son síncronos o asíncronos y dónde se aplica la seguridad.




*Línea continua: llamada síncrona. Línea punteada: paso asíncrono, en otra transacción. S1 a S5: puntos donde se aplica la seguridad (**sección 7**).*

La decisión que sostiene el diseño es separar la recepción del procesamiento con un Platform Event en medio; su porqué y su costo están en «Del problema al diseño» y en la sección 9.

# 4. Vista lógica en capas

| | |
|---|---|
| **Presentación** | App Nova Casa Telemetry · Páginas de edificio, activo e intervención<br>`novaCasaActivosOperador` `novaCasaActivoEstado` `novaCasaEdificioResumen` `novaCasaIntervencion` `novaCasaCrearIntervencion` `novaCasaEtiquetas` `novaCasaIlustracion` |
| **Controladores** | `ActivosOperadorController` `IntervencionOperadorController` `IngestaController`<br>*Fachada para la interfaz: devuelve vistas propias, nunca registros completos* |
| **Dominio** | `SenialSensorTrigger` `SenialSensorHandler` `IntervencionService` `OpenCaseCheck` `UmbralTrigger` `UmbralValidador`<br>*Reglas del negocio: clasificación, estado actual, intervenciones* |
| **Integración** | `TelemetriaIngestaProgramada` `TelemetriaIngesta` `SimuladorClient` `SesionSimulador` `CatalogoService` *Habla con el simulador y publica el evento* |
| **Datos** | `Account` `Asset` `Case` `Lectura_Vigente__c` `Log_Senial__c` `Umbral__c` `Senial_Sensor__e` |
| **Plataforma** | Event Bus · Platform Cache · Apex asíncrono y programador<br>Named y External Credential · Base de datos |

**SEGURIDAD TRANSVERSAL**

Permission sets por persona

OWD privado (Asset y Lectura heredan del padre; Umbral__c es lectura/escritura) y jerarquía de roles

with sharing y USER_MODE en la consulta

Permisos personalizados en cada acción

Identidad de ingesta separada

*Capas de la solución. Cada capa usa las inferiores y las de su misma capa, nunca las superiores; la seguridad atraviesa todas.*

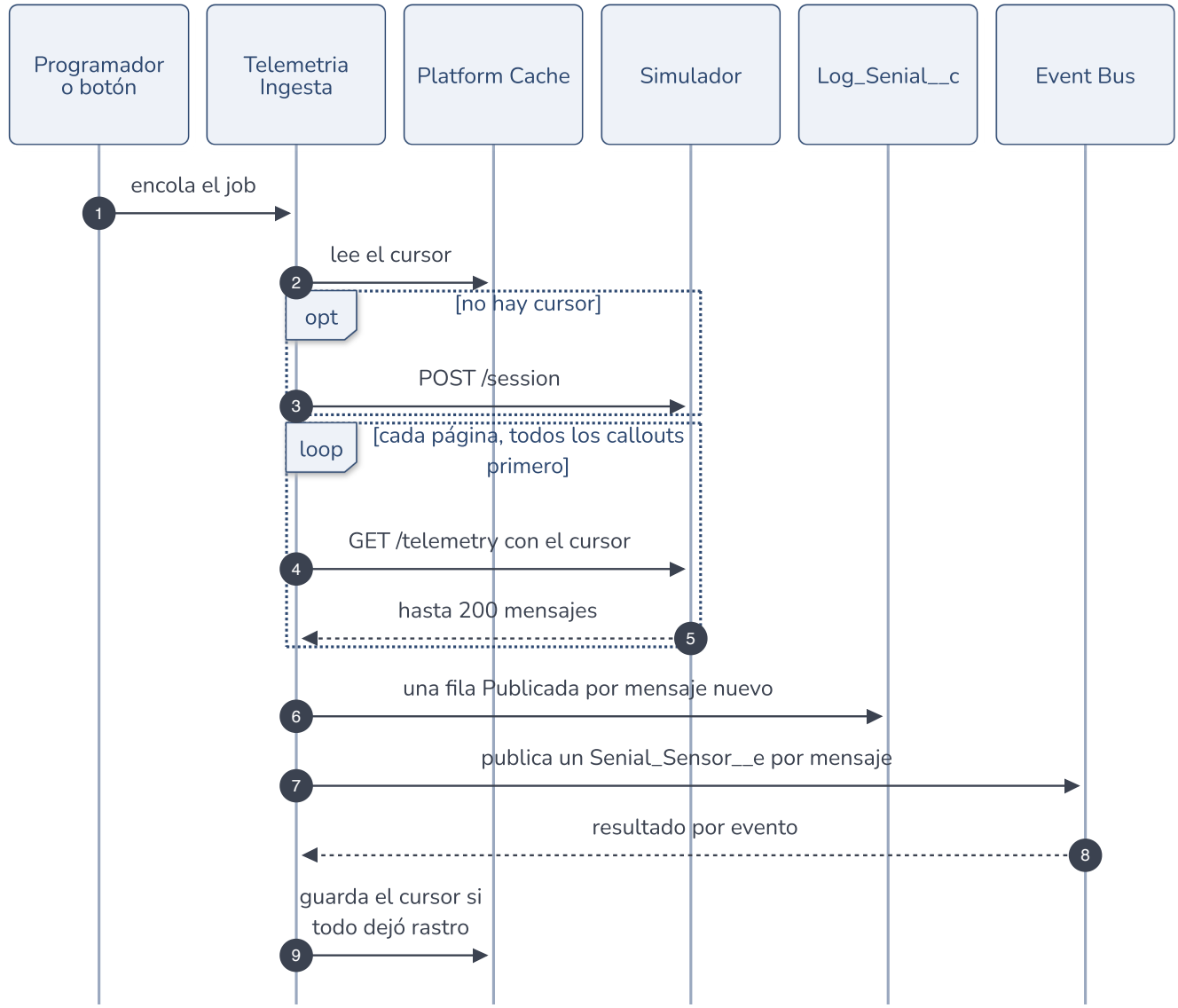
| Capa | Pieza | Qué hace | Modo | Corre como |
|---|---|---|---|---|
| Presentación | App Nova Casa Telemetry; LWC `novaCasaActivosOperador`, `novaCasaActivoEstado`, `novaCasaEdificioResumen`, `novaCasaIntervencion`, `novaCasaCrearIntervencion`, `novaCasaEtiquetas`, `novaCasaIlustracion`; ruta guiada del estado del caso | Pantalla del operador y páginas de edificio, activo e intervención. | Síncrono | El usuario |
| Controladores | `ActivosOperadorController`, `IntervencionOperadorController`, `IngestaController` | Lectura de la pantalla, acciones sobre intervenciones e inicio de la ingesta. | Síncrono | El usuario, con su acceso |
| Dominio | `SenialSensorTrigger`, `SenialSensorHandler`, `IntervencionService`, `OpenCaseCheck`, `UmbralTrigger`, `UmbralValidador`; regla que exige un comentario al cambiar el estado de un caso | Clasifica, decide la lectura vigente, abre o enlaza la intervención y valida los límites. | Asíncrono (suscriptor) y síncrono (validaciones) | Usuario de integración; el usuario en las validaciones |
| Integración | `TelemetriaIngestaProgramada`, `TelemetriaIngesta`, `SimuladorClient`, `SesionSimulador`, `CatalogoService` | Consulta al simulador, guarda el cursor, escribe la fila Publicada, publica el evento y carga el catálogo. | Asíncrono (Queueable y Scheduled Apex); `CatalogoService` es síncrono, `with sharing` y en modo usuario | Quien agenda o inicia; `CatalogoService` corre como quien lo ejecuta |

| Capa | Pieza | Qué hace | Modo | Corre como |
|---|---|---|---|---|
| Datos | `Account`, `Asset`, `Case`, `Lectura_Vigente__c`, `Log_Senial__c`, `Umbral__c`, `Senial_Sensor__e` | Edificios, activos, intervenciones, lectura vigente por activo y medición, registro de cada señal, límites y evento. | - | - |
| Plataforma | Event Bus, Platform Cache (partición `NovaCasaSimulador`), Apex asíncrono y programador, Named y External Credential `Nova_Casa_Simulator` | Servicios de Salesforce en los que se apoya la solución. | - | - |

Se usan objetos estándar donde alcanzan: el edificio es `Account`, el activo es `Asset` y la intervención es `Case`. Los objetos personalizados existen porque no hay uno estándar para lo que guardan. El suscriptor es Apex y no Flow porque la frontera formativa pide un suscriptor programático preparado para colecciones.

# 5. Vista de comportamiento

## 5.1 Recepción de señales



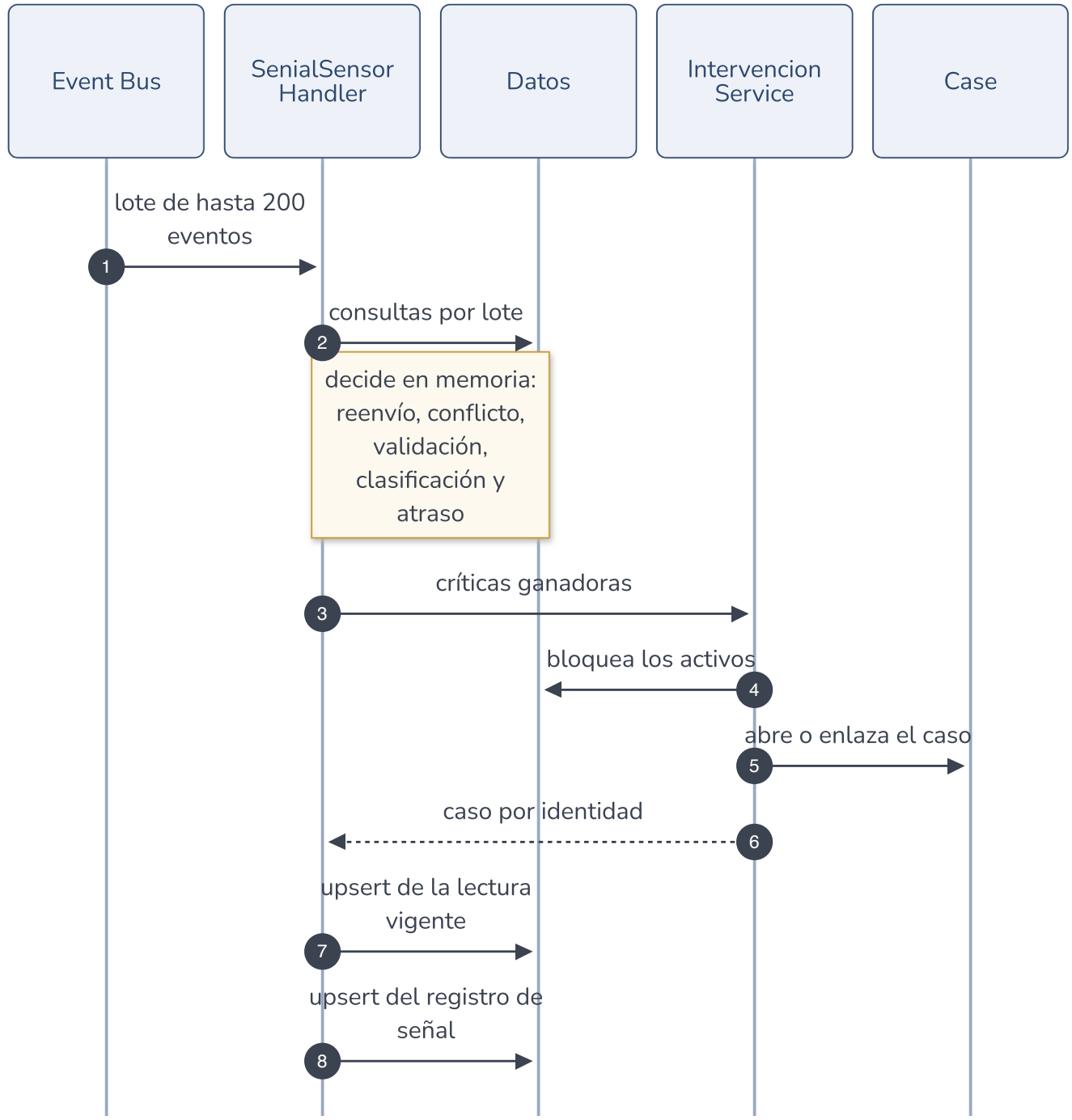

*Secuencia de una corrida de ingesta.*

- Un job programado (12 jobs, uno cada 5 minutos) o el botón «Traer señales» encolan la ingesta; una comprobación previa evita lanzar otra mientras hay una en cola o corriendo.
- Si hay un cursor guardado para el escenario, la corrida continúa donde quedó la anterior; si no, abre una sesión (D025).
- Un mismo job hace primero todos los callouts y después todas las escrituras, porque Salesforce no permite un callout después de una escritura sin confirmar. Ante 429 o 5xx reintenta la página; ante 409 o 410 reabre la sesión una vez (D023).

- Por cada mensaje nuevo escribe una fila Publicada con su identidad, su fecha de origen y la de publicación, y después publica el evento. Un mensaje cuya identidad ya tiene fila se publica igual, para que el suscriptor cuente el reenvío o el conflicto; solo al releer un flujo ya terminado se omiten los que ya tienen resultado (D040). Si la publicación falla, la fila que esa corrida creó queda Fallida; si falla el alta de la fila, ese mensaje no se publica. Un mensaje sin el mínimo para publicarse, por ejemplo sin identidad válida, queda Rechazada con su motivo.
- El cursor se guarda en Platform Cache solo si todos los mensajes dejaron su rastro; si no, la siguiente corrida relee esas páginas; la identidad evita filas e intervenciones duplicadas (los mensajes que ya tienen fila se vuelven a publicar y cuentan como reenvío).

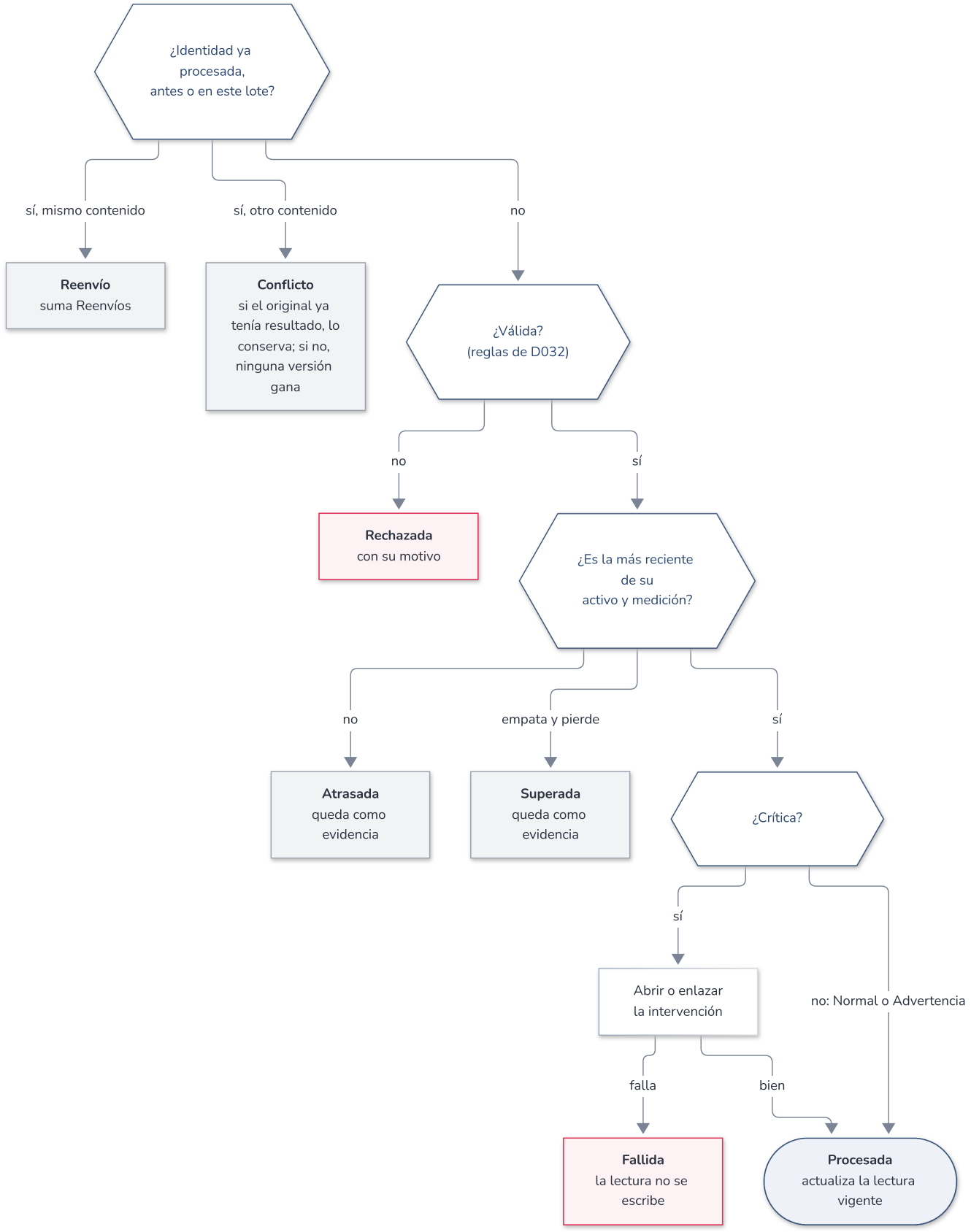
## 5.2 Procesamiento de un lote

- La plataforma entrega los eventos al trigger en lotes de hasta 200, como el usuario de integración configurado en el `PlatformEventSubscriberConfig`.
- El handler hace un número fijo de consultas por lote (identidades, activos, lecturas vigentes y umbrales) y decide todo en memoria. La huella del contenido es un SHA-256 sin los campos de entrega.
- Por activo y tipo de medición gana la señal de fecha de origen más reciente; con la misma fecha, la de mayor severidad; después, la de `messageId` menor y, por último, la de `source` menor (D003, D011). La fecha se compara al segundo y el orden de llegada nunca decide.
- Solo las críticas ganadoras llegan a `IntervencionService`, que bloquea los activos y abre la intervención o enlaza la señal a la que ya está abierta (D039). Al final se guardan la lectura vigente y el resultado de cada señal con su hora de procesamiento.
- La severidad del activo es la mayor de sus lecturas vigentes, una por tipo de medición (resumen de `Lectura_Vigente__c` hacia `Asset`): una temperatura normal no tapa una presión crítica del mismo activo.



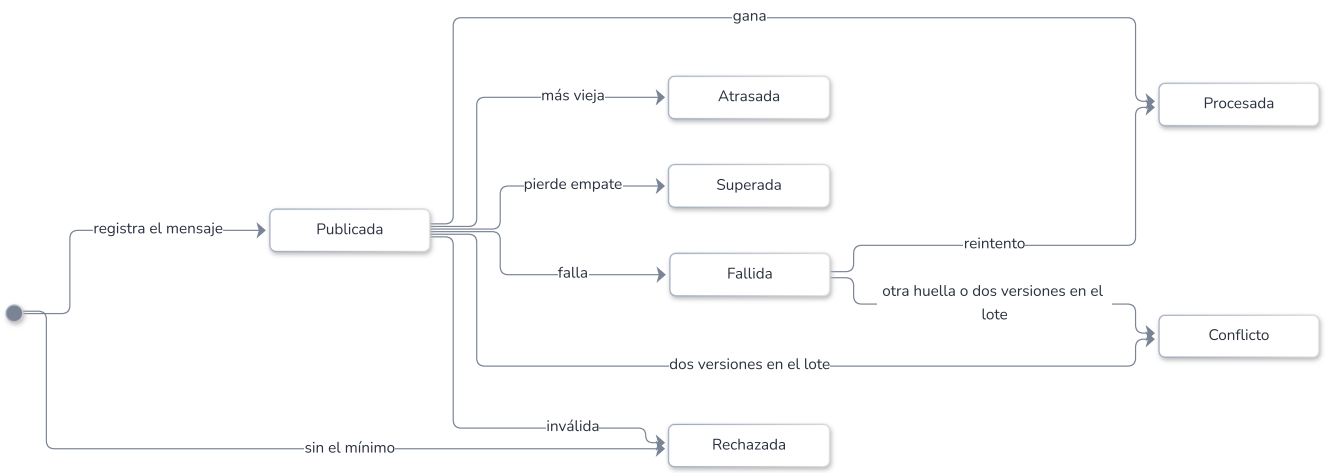

*Secuencia del suscriptor para un lote de eventos.*

## 5.3 Decisión por señal




*Flujo de decisión que aplica el handler a cada evento del lote. Si falla la intervención o el guardado de la lectura, la señal queda Fallida.*

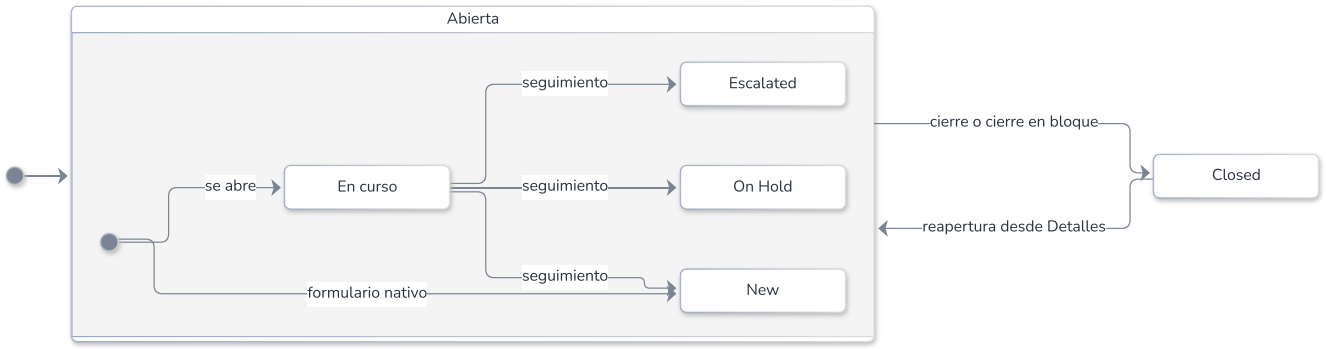
## 5.4 Ciclo de vida de una señal




*Estados del resultado de una señal en `Log_Senial__c`.*

Procesada, Atrasada, Superada, Rechazada y Conflicto son resultados finales. Un reenvío idéntico o un conflicto que llega después no cambian el resultado: se cuentan aparte, en el conteo de reenvíos o en el de conflictos con el detalle de lo que trajo la entrega (D047). Publicada y Fallida son los únicos que se vuelven a procesar: la siguiente entrega se decide como la primera vez, así que puede terminar en cualquier resultado; en Conflicto si la fila ya guardaba otra huella (Fallida) o si las dos versiones llegan en el mismo lote.

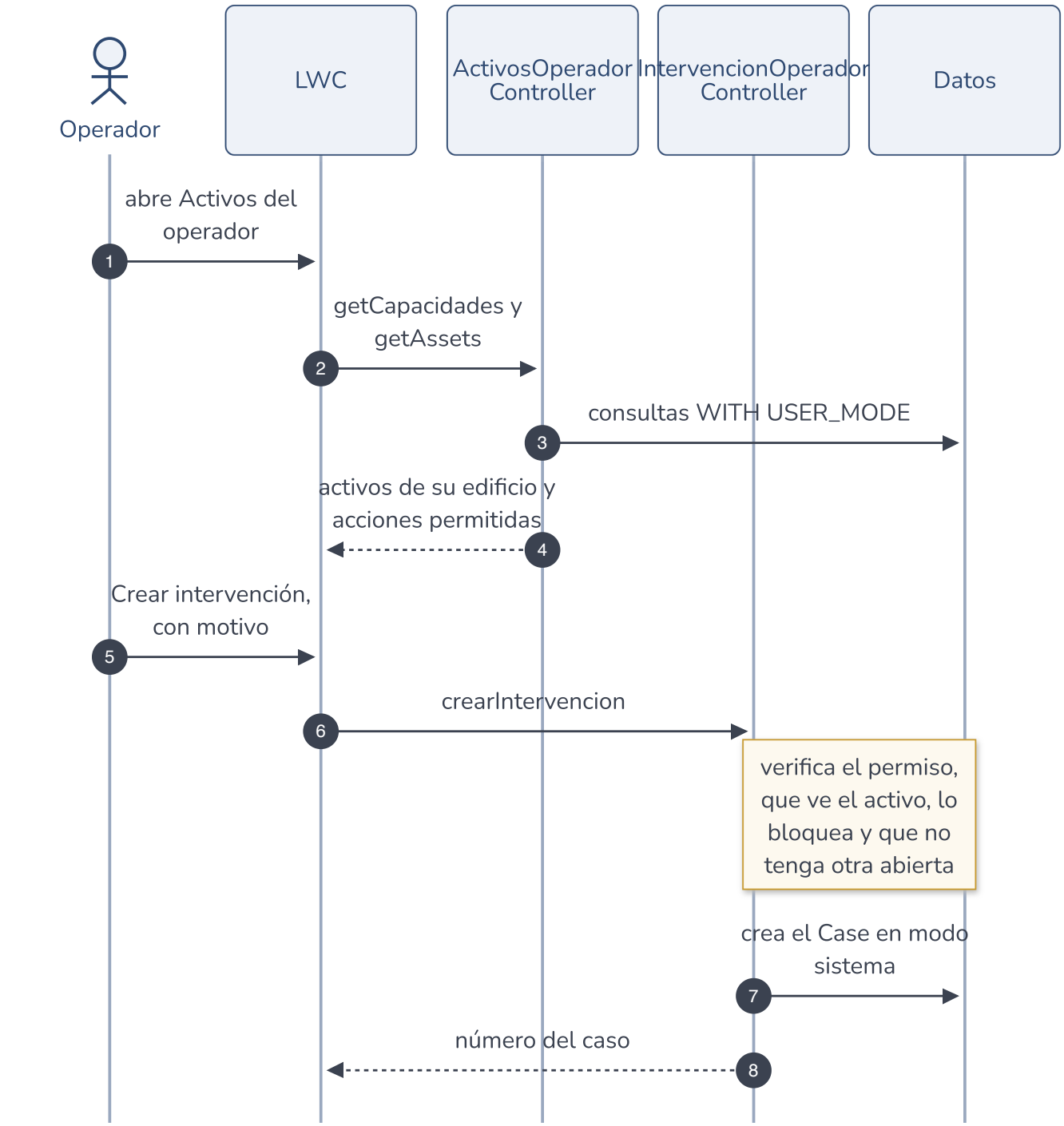
## 5.5 Ciclo de vida de una intervención




*Estados de una intervención (`Case.Status`). Entre los estados abiertos se pasa de cualquiera a cualquiera, siempre con un comentario.*

Una intervención de telemetría se abre En curso cuando llega una crítica vigente de un activo sin intervención de telemetría abierta; una persona la abre a mano si el activo no tiene ninguna abierta. Una intervención manual abierta no impide que la telemetría abra la suya. Mientras siga abierta, cada crítica nueva del activo se enlaza a la de telemetría y actualiza la última crítica registrada si es más reciente (D039). Cada cambio de estado exige un comentario, y la página del caso guía cada paso con una ruta. La telemetría nunca reabre una intervención cerrada: la siguiente crítica vigente abre una nueva.

## 5.6 Consulta y acción del operador

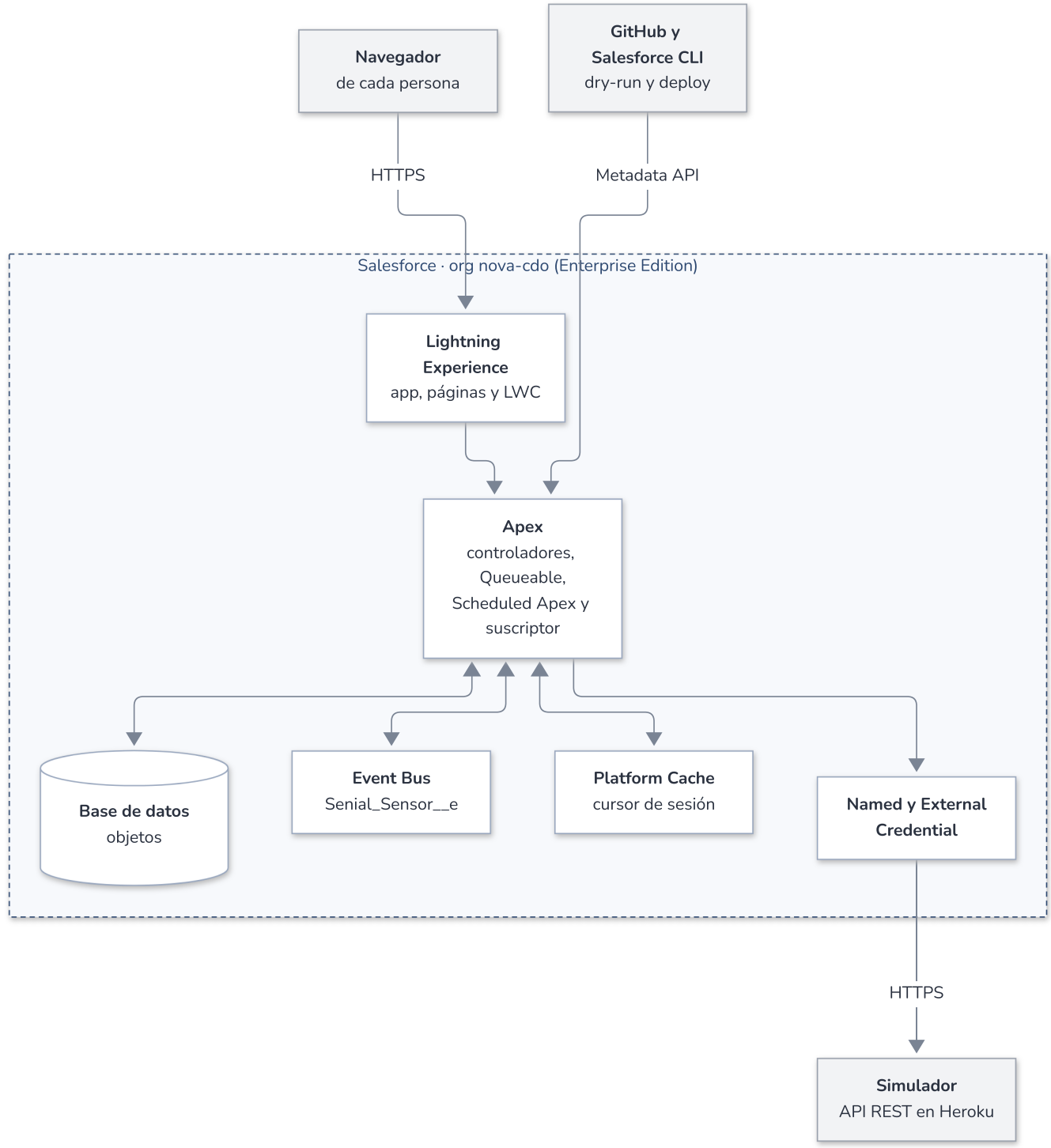



*Secuencia de la pantalla del operador y de la creación de una intervención manual.*

## 5.7 Síncrono y asíncrono

| Interacción | Modo | Por qué |
|---|---|---|
| Job programado o botón, hacia `TelemetriaIngesta` | Asíncrono (Queueable) | Los callouts y el volumen quedan fuera de la transacción de la persona. El botón responde enseguida si la corrida arrancó. |
| `TelemetriaIngesta` hacia el simulador | Callout síncrono dentro del job | Cada página se pide y se espera; los reintentos se encolan como otro job. |
| Publicación hacia procesamiento | Asíncrono (bus de eventos) | Publicar no es procesar: cada momento tiene su fecha en el registro de la señal. |
| Procesamiento hacia almacenamiento | Síncrono, en la transacción del suscriptor | La lectura, la intervención y el resultado se deciden y se guardan en el mismo lote. |
| Pantalla hacia controladores | Síncrono ( `@AuraEnabled` ) | La persona ve el resultado en el momento; «Actualizar» vuelve a pedir los datos. |
| Guardar un umbral o cambiar el estado de un caso | Síncrono (validación antes de guardar) | Un rango contradictorio o un cambio sin comentario se rechazan en el momento, con su mensaje. |

# 6. Vista de despliegue

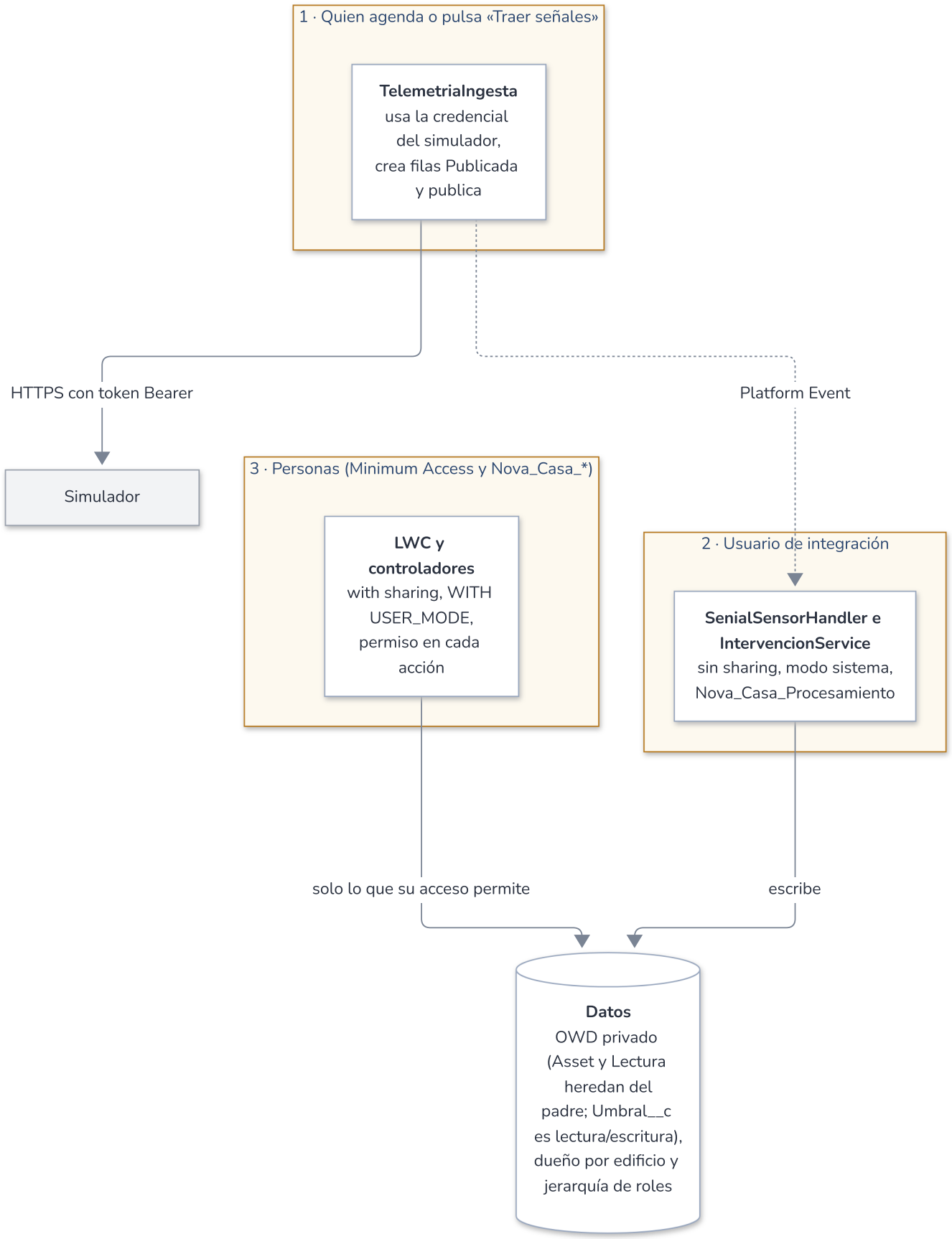



*Nodos de ejecución y conexiones.*

| Nodo | Qué corre o guarda |
| --- | --- |
| Navegador | Lightning Experience con la app Nova Casa Telemetry; los LWC corren en el cliente bajo Lightning Web Security. |
| Org `nova-cdo` | Apex síncrono (controladores y validación de umbrales) y asíncrono (Queueable, 12 jobs de Scheduled Apex y el suscriptor del evento); las reglas de validación del Case; el Event Bus; la base de datos; la partición de Platform Cache con el cursor; y la Named y External Credential con el token del simulador. |
| Heroku | El simulador del proveedor, consumido por HTTPS con token Bearer. |
| GitHub y Salesforce CLI | El proyecto Salesforce DX y el despliegue por Metadata API. |

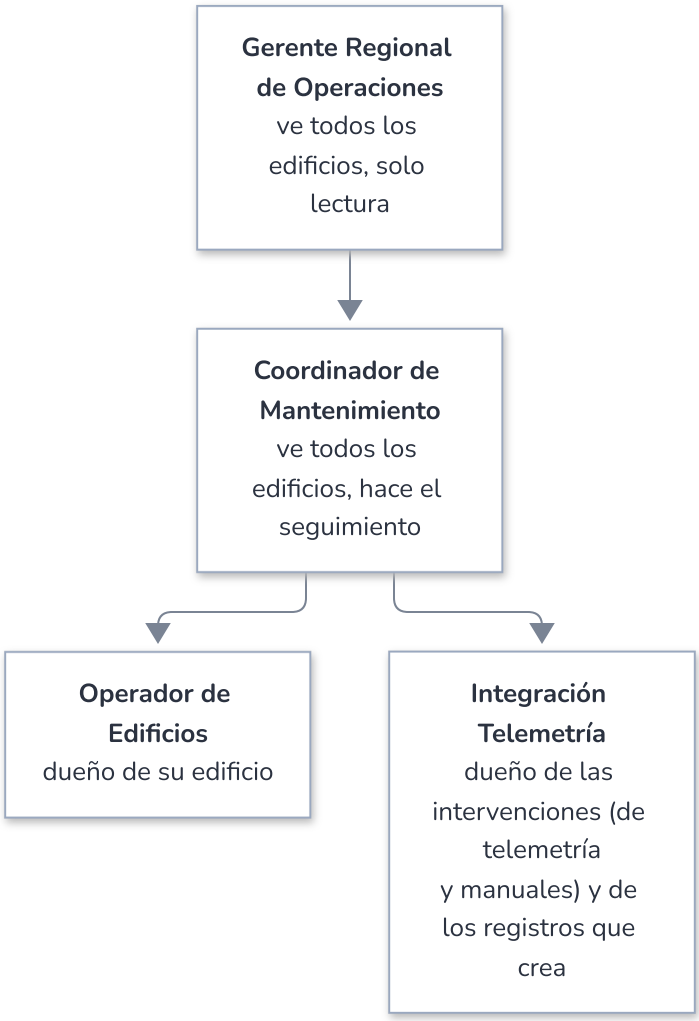
# 7. Seguridad

## 7.1 Identidades y fronteras de confianza




*Tres identidades con permisos distintos. La ingesta nunca depende de los permisos de un operador.*

## 7.2 Jerarquía de roles




*Cada edificio pertenece a un operador; coordinador y gerente lo ven por la jerarquía, sin View All.*

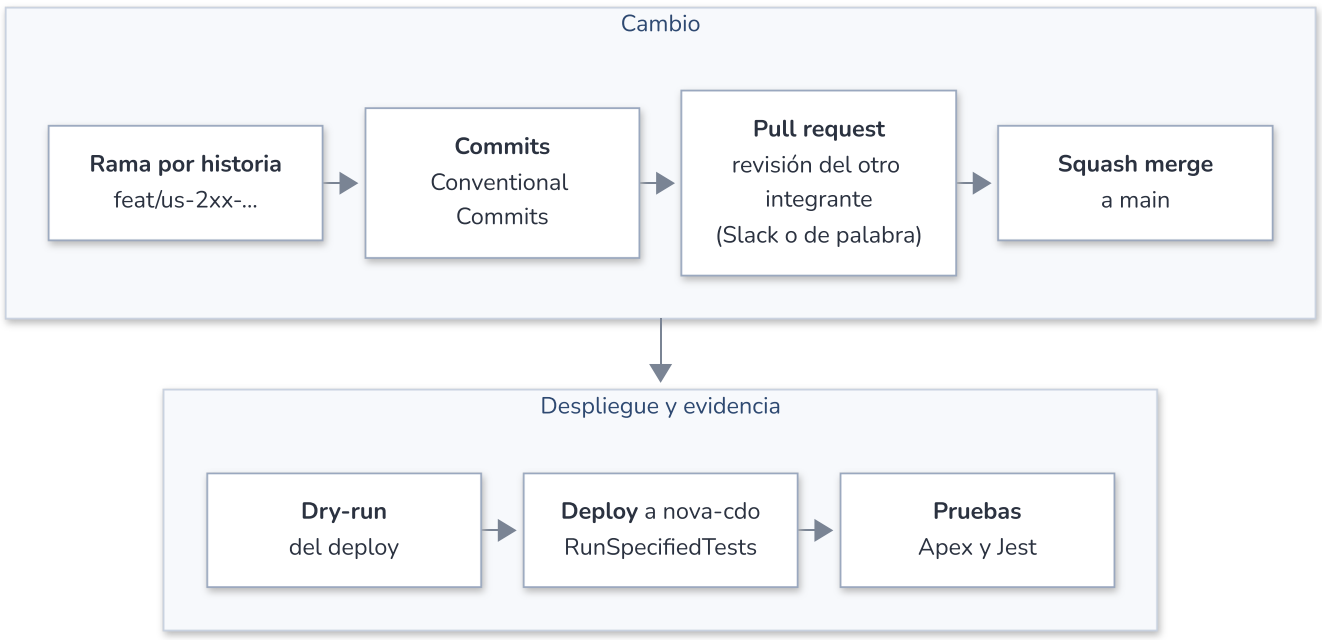
## 7.3 Dónde se aplica la seguridad

| Punto | Dónde | Cómo |
|---|---|---|
| S1 | Credencial del simulador | El token Bearer vive en la External Credential (principal `NovaCasaSimulatorPrincipal`), solo lo ingresa un administrador en Setup y solo lo usan los usuarios con `Nova_Casa_Simulator_Integration`. El cursor vive en Platform Cache y en el estado privado del job siguiente: nunca en registros, logs, mensajes de error ni respuestas a la pantalla. |

| Punto | Dónde | Cómo |
|---|---|---|
| S2 | Identidad de la ingesta | Traer y publicar corre como quien agenda la ingesta o pulsa el botón, que necesita la credencial del simulador y permiso para crear `Senial_Sensor__e`. Procesar corre como el usuario de integración (perfil Minimum Access, rol Integración Telemetría, permission set `Nova_Casa_Procesamiento`). `TelemetriaIngesta`, `SenialSensorHandler` e `IntervencionService` corren sin sharing y en modo sistema (D017). |
| S3 | Consulta | Account y Case son privados; Asset y Lectura_Vigente__c heredan del padre. Cada edificio pertenece a un operador, y coordinador y gerente lo ven por la jerarquía de roles (D022). Los controladores son `with sharing` y consultan `WITH USER_MODE`, que respeta permisos de objeto y de campo: un operador no obtiene datos de otro edificio ni llamando a Apex directamente (BR-208), salvo las excepciones conocidas que describe S5. |
| S4 | Acciones | Crear intervención, seguimiento y «Traer señales» exigen cada una su permiso personalizado, verificado en el servidor antes de actuar; la pantalla solo oculta los botones. En la página del caso, el cambio de estado desde Detalles depende además del permiso de edición sobre Case, que tienen el coordinador, el administrador y el usuario de integración (este sin acceso a la interfaz), y de la regla que exige el comentario. Pasar un caso a Closed exige además el permiso de seguimiento por cualquier vía (Detalles, API o Apex): lo impone la regla `Cierre_Exige_Permiso_Seguimiento` (D052; ya desplegada). El permission set del administrador administra la telemetría, no los usuarios ni la seguridad de la org (D050). |

| Punto | Dónde | Cómo |
|---|---|---|
| S5 | Investigación y excepciones | Solo el administrador ve `Log_Senial__c`. En la ruta de las personas hay seis operaciones en modo sistema, cinco lecturas y una escritura, cada una con su control previo: los valores recientes de un activo (D024), con medición, valor y unidad solo de lecturas aceptadas (D056), las señales enlazadas a una intervención (D043) y la señal que abrió cada intervención (D045), que primero comprueban en modo usuario que la persona ve el registro y no devuelven campos técnicos; la comprobación de si el activo ya tiene una intervención abierta, que solo responde sí o no (D021); la búsqueda del usuario de integración que será dueño del caso manual, que solo devuelve un Id (D053); y el alta del caso manual, detrás de su permiso y con el activo bloqueado (D020). Además, «Traer señales» consulta en modo sistema si ya hay una ingesta en curso, que también solo responde sí o no (D040), y la validación de umbrales lee las demás reglas sin aplicar permisos de campo (D018). La revisión de seguridad del 26 de octubre dejó tres excepciones (D020, D024 y D045); D053 (28 de octubre) cerró dos: el caso manual lo crea el usuario de integración y sigue al edificio, y la consulta de una intervención no devuelve el Id del activo ni la última crítica si la persona no ve el activo. Queda una: un caso sin activo no pasa la comprobación de activo al leer sus señales. |

# 8. Vista de desarrollo




*Flujo de un cambio, desde la rama hasta la evidencia.*

Todo el código y la metadata salen del repositorio y la regla del equipo es no cambiar nada a mano sin versionarlo; los usuarios, los jobs, el token y la carga del catálogo son datos de la org. Cada cambio vive en su rama, entra a `main` por un pull request que revisa el otro integrante (la revisión se hace por Slack o de palabra y no deja comentarios en GitHub) y se valida con un dry-run antes del despliegue.

| Carpeta | Contenido |
|---|---|
| `force-app/main/default/classes` y `triggers` | 12 clases Apex, 19 clases de prueba y 2 triggers. |
| `lwc` | Los componentes de la interfaz y sus pruebas Jest. |
| `objects`, `standardValueSets` | Objetos, campos, reglas de validación, vistas de lista y valores de los picklists del caso. |
| `permissionsets`, `customPermissions`, `roles`, `profiles` | El modelo de acceso. |
| `applications`, `tabs`, `flexipages`, `layouts`, `pathAssistants` | La app, sus pestañas, las páginas de registro y la ruta guiada del caso. |
| `namedCredentials`, `externalCredentials`, `cachePartitions`, `platformEventSubscriberConfigs` | La integración con el simulador y la configuración del suscriptor. |
| `docs/` y `scripts/` | Decisiones, entrega y matriz de acceso; scripts para programar la ingesta y revisar la org. |

## 9. Decisiones de diseño

Son decisiones del equipo, registradas con su alternativa y su trade-off en `docs/decisiones.md`. D039 se confirmó de palabra con el facilitador, sin comentario escrito en MDSS, y John la aprobó por escrito como responsable el 28 de octubre de 2026. D005, D006, D011, D019, D021, D024, D031, D032 y D034, la excepción en modo sistema de D020 y el OWD de D022 son decisiones nuestras que John aprobó como responsable el 28 de octubre de 2026, en lugar de la confirmación escrita del facilitador; el facilitador no las ha confirmado por escrito.

| Decisión | Alternativa descartada | Trade-off |
|---|---|---|
| **D001.** La intervención es un `Case`. | Work Order. | Viene habilitado y reutiliza estados, colas, permisos y vistas; se pierden los campos de mantenimiento de Work Order, que además requieren Field Service. |
| **D013 y D031.** Nuestro Apex trae las señales y publica el Platform Event; una fila Publicada se escribe antes de cada publicación. | Que el simulador empuje los eventos (su contrato no lo ofrece) o una sola Queueable que recorra todas las páginas en un bucle (D013). | Una pieza más, a cambio de que recepción y reglas cambien por separado y de que publicar y procesar tengan evidencia propia. Es la forma de recibir Platform Events reales de un simulador que solo se consulta por API. |
| **D004.** Los límites son el objeto `Umbral__c`. | Custom Metadata Type. | El coordinador los edita sin permisos de configuración; a cambio, los registros no viajan como metadata entre entornos. |
| **D011.** La lectura vigente es un objeto propio, una fila por activo y medición, y el handler decide reenvío, conflicto y atraso. | Campos en `Asset`, como proponía el borrador de Discovery (una temperatura pisaría una presión), y escribir la lectura para revertirla si falla el caso. | No queda una lectura crítica sin su intervención y el lote sigue con éxito parcial; si la crítica ganadora falla, las que venció quedan Atrasada (o Superada si empataban en la hora) y no se promueve la siguiente. Leer «atrasada» también dentro de un lote es una interpretación nuestra del acuerdo, que el facilitador no ha confirmado por escrito; John la aprobó como responsable el 28 de octubre de 2026, en lugar de esa confirmación. |
| **D017.** Un usuario de integración con su propio permission set. | Automated Process o reutilizar el permission set de administrador. | Es auditable, tiene rol (coordinador y gerente ven sus casos) y se puede probar; cuesta mantener a mano sus permisos de campo. |
| **D022.** Compartición privada, un operador dueño por edificio y jerarquía de roles. | View All para el gerente. | El acceso es fácil de explicar y de probar; un edificio nuevo queda a nombre de quien carga el catálogo hasta que se asigna a un operador. |

| Decisión | Alternativa descartada | Trade-off |
|---|---|---|
| **D025.** El cursor vive en Platform Cache y un job lee varias páginas. | Cursor en un campo o Custom Setting (un secreto en datos) y un job por página. | La corrida sigue donde quedó y no repite mensajes; depende de Platform Cache y de un estado oculto que la pantalla explica con «Reiniciar sesión». |
| **D039.** Una intervención de telemetría abierta por activo, actualizada con la crítica más reciente. | Una por mensaje crítico único (el alcance base), una por activo y medición, o cerrarla sola al volver a normal. | El activo no acumula casos; el caso muestra la situación y no cada mensaje. La garantía base se sigue cumpliendo. El facilitador la confirmó de palabra, sin comentario escrito en la tarjeta de MDSS; la limitación del diseño base se explica en la **Documentación Funcional, sección 5**. |
| **D040.** La ingesta programada corre cada 5 minutos con 12 jobs. | Un cron «cada 5 minutos» (Salesforce no lo admite), un job que se reprograme solo o un Queueable con retraso. | Recepción continua; mientras los jobs estén agendados no se pueden desplegar las clases de ingesta, así que hay que desprogramar, desplegar y volver a programar. |
| **D047.** Un conflicto sobre una señal ya procesada no cambia su resultado: se cuenta aparte, con el detalle de lo que trajo la entrega. | Marcar la fila como Conflicto y perder el resultado, el motivo y la hora del original. | La evidencia del original se conserva y el conflicto queda visible en su vista de lista; si las dos versiones llegan antes de que el original se aplique, ninguna gana y la fila queda Conflicto. |

## 10. Atributos de calidad

Cada atributo se expresa como un escenario concreto, la táctica de arquitectura que lo resuelve y la prueba automatizada que falla si esa táctica se rompe.

| Atributo | Escenario | Táctica | Prueba que lo verifica |
|---|---|---|---|
| Integridad | El proveedor reenvía un mensaje ya procesado, o la misma clave con otro contenido. | Identidad única y huella del contenido: el reenvío solo se cuenta y el conflicto no cambia el estado. | `unReenvioIgualSoloSumaReenviosSinTocarLecturaNiCase`, `mismaIdentidadConOtroContenidoEsConflictoSinCambiarEstado` |
| Consistencia temporal | Llega una lectura más vieja que la vigente, o dos con la misma hora. | Comparación por activo y medición con desempate determinista. | `unaSenialMasAntiguaQueLaVigenteQuedaAtrasadaAunqueSeaCritica`, `elDesempateD003NoDependeDelOrdenDeLlegada` |
| Rendimiento | Llega un lote de 200 eventos. | Decisión en memoria con un número fijo de consultas y escrituras. | `doscientosEventosEnUnLoteUsanUnNumeroFijoDeConsultasYDml` |
| Tolerancia a fallos | Un evento trae datos inválidos o falla la creación de su intervención. | Aislamiento por evento y éxito parcial. | `unLoteMixtoDe200SenialesSeparaValidasEInvalidasConUnNumeroFijoDeConsultasYDml`, `unaExcepcionEnUnEventoNoTumbaElLote`, `siFallaElCaseLaIdentidadQuedaFallidaSinLecturaYLasDemasSeProcesan` |
| Disponibilidad de la integración | El simulador responde 429 o 5xx, o vence la sesión. | Reintentos con espera, reapertura de sesión y cursor que solo avanza si todo quedó registrado. | `TelemetriaIngestaTest`, `SimuladorClientTest` |
| Seguridad | Un operador pide a Apex los datos de otro edificio. | Controladores `with sharing`, consultas `WITH USER_MODE`, compartición privada y permisos verificados en el servidor. | `operatorDoesNotSeeAnotherOperatorsBuildingButGerenteSeesBoth`, `SeguridadTest`, `AccesoFueraDeJerarquiaTest` |
| Independencia de la ingesta | La ingesta corre con un usuario sin permisos de operador. | Identidades separadas y modo sistema en el procesamiento. | `laIngestaPersisteConUnUsuarioSinPermisosDeOperador`, `PermisosProcesamientoTest` |

| Atributo | Escenario | Táctica | Prueba que lo verifica |
|---|---|---|---|
| Modificabilidad | El coordinador cambia un límite. | Reglas como datos, con validación de rangos al guardar. | `editarElLimiteDeUnUmbralCambiaLaClasificacionSinTocarElHistorico`, `operadorGTUnValorIgualAlLimiteNoLoCumpleYElSiguienteSi`, `UmbralValidadorTest` |
| Trazabilidad | El administrador investiga una señal rechazada. | Registro persistente por señal con motivo, fechas y acción de reintento. | `LogAccionReintentoTest` |
| Usabilidad | El operador debe reconocer qué necesita atención. | Severidad con color, glifo y texto; estados de carga, vacío y error. | Pruebas Jest de `novaCasaActivosOperador`, `novaCasaActivoEstado` y `novaCasaEtiquetas` |

## Límites operativos

- **Lotes:** hasta 200 eventos por lote del suscriptor y hasta 200 mensajes por página del simulador.
- **Páginas:** una corrida desde «Traer señales» lee hasta 25 páginas; desde la consola de desarrollo, hasta 100; cada corrida programada lee 5. La cadena de jobs se corta en 27 como protección.
- **Reintentos:** hasta 3 intentos por página ante 429 o 5xx, con 1 y 2 minutos de espera; la sesión se reabre una vez ante 409 o 410.
- **Entregas concurrentes:** la identidad es única a nivel de base de datos, así que un mismo mensaje nunca genera dos filas ni dos intervenciones; una comprobación previa evita lanzar otra ingesta mientras hay una en cola o corriendo, y el activo se bloquea antes de abrir o enlazar su intervención. Una prueba Apex no ejecuta dos transacciones a la vez, así que estas carreras están resueltas por diseño, con la restricción única y el bloqueo.

## Pruebas

Cada clase Apex tiene su clase de prueba, y las pruebas fallan si se rompe el comportamiento que afirman. La corrida completa del 28 de octubre de 2026 en la org dio 460 de 460 pruebas aprobadas en 19 clases de prueba, con 90 % de cobertura en la org (D056). Los componentes principales de la interfaz tienen además pruebas Jest.

## 11. Riesgos operativos y mitigación

| Riesgo | Dependencia | Mitigación | Siguiente acción |
|---|---|---|---|
| El contrato del simulador no documenta qué responde al vencer una sesión o al limitar la tasa. | Lo que el simulador real devuelve al vencer una sesión o limitar la tasa: 409, 410 y 429 son una suposición, no contrato (D023). | Reintentos ante 429 y 5xx, reapertura de sesión ante 409 y 410, y continuación desde el último cursor válido (D023, D025). | Observar en vivo un reintento por 429/5xx y una sesión reabierta y registrar qué devuelve el simulador: hasta ahora no se ha visto ninguno (`docs/entrega.md`, HP-05). La pregunta al facilitador está redactada para la tarjeta de US-201, pero ya no bloquea ninguna decisión. |
| El simulador reutiliza claves de mensaje entre sesiones. | El simulador repite `messageId` entre sesiones (D040). | La identidad es única: un mensaje repetido se reconoce como reenvío o conflicto y nunca crea un estado nuevo. Las corridas de evidencia parten de una org sin datos de corridas anteriores. | Desprogramar la ingesta antes de limpiar la org o de correr otro escenario (D040) y limpiar antes de cada corrida de evidencia (`docs/entrega.md`). |
| Platform Cache sin capacidad o con la entrada desalojada. | La capacidad de Platform Cache de la org: la partición pide 1 MB (D025). | La corrida abre una sesión nueva, lo indica en pantalla y la identidad evita duplicados (D025). | Ajustar la capacidad de la partición si la org la limita. Si la caché se vacía, la siguiente corrida abre una sesión nueva sin avisar la causa: la pantalla solo dice «abrió una sesión nueva», y el aviso «sin caché de plataforma» aparece únicamente cuando la caché no está disponible (D025). |

| Riesgo | Dependencia | Mitigación | Siguiente acción |
|---|---|---|---|
| Los jobs programados son datos de la org, no metadata. | El usuario que agenda los jobs: corren como él, y si se desactiva o pierde su permission set cada corrida falla (D040). | Se crean y se quitan con scripts versionados: `scripts/apex/programar-ingesta.apex` se ejecuta en cada org nueva y `desprogramar-ingesta.apex`, antes de limpiar o desplegar. | Ejecutar `programar-ingesta.apex` en cada org nueva, con un usuario que tenga `Nova_Casa_Simulator_Integration` y Create sobre `Senial_Sensor__e`. |
| Salesforce no permite desplegar una clase agendada ni las que referencia. | El ajuste de despliegue de la org que lo permitiría es metadata compartida y se deja apagado (D040). | Procedimiento de despliegue documentado: desprogramar, desplegar y volver a programar (D040). | Desprogramar antes de desplegar esas clases y programar después. |
| La org de trabajo es una org de prueba que vence el 19 de noviembre de 2026. | Que haya una org nueva donde desplegar. Los usuarios, los dueños de los edificios, el token, el catálogo y los jobs son datos de la org, no metadata. | Ese día desaparece la org y, con ella, los jobs programados. El código y la metadata se vuelven a desplegar en una org nueva; los usuarios y los dueños de los edificios se rehacen (`docs/acceso.md`), el token y el catálogo se vuelven a cargar y los jobs se agendan otra vez con el script (D040). | Seguir la «Puesta en marcha en una org nueva» (**Documentación Técnica, sección 13**) antes de esa fecha. |
| En una org de prueba, el programador de Salesforce dispara cada franja con unos minutos de retraso. | El programador de la org de prueba. | No cambia el resultado, solo el momento. Para una demostración, la ingesta se puede iniciar a pedido con «Traer señales». | Ninguna: no cambia el resultado, solo el momento. |

## 12. Evolución posible

- Notificaciones automáticas al coordinador cuando un activo pasa a crítico.

- Derivar la intervención a Field Service (orden de trabajo) si Nova Casa lo habilita.
- Particionar la suscripción del evento si el volumen crece; hoy los lotes se procesan de uno en uno.